

IronMind Coach

API Technical Reference

REST API for High-Performance Triathlon Training & Recovery

Version 1.0 | March 2026

Base URL (Live)	https://osamakadhem.pythonanywhere.com/api/v1
Base URL (Local)	http://localhost:8000/api/v1
Auth Header	Authorization: Bearer or X-API-KEY: <code>ironmind_secret_2026</code>
Content-Type	application/json
Rate Limit	120 req/min per IP (30 req/min on /health)
Swagger UI	https://osamakadhem.pythonanywhere.com/docs
ReDoc	https://osamakadhem.pythonanywhere.com/redoc

Table of Contents

1. Authentication	2
2. Endpoint Reference	2
3. Athlete Management	2
4. Training & Wellness Logs	3
5. Performance Analytics	4
6. Advanced Features	5
7. Readiness Algorithm	6
8. Validation Rules	6
9. Error Reference	7

IronMind Coach

Evidence-Based Triathlon Training API

Grounded in 6 peer-reviewed citations · Validated against 1,000 real athlete records

osamakadhem.pythonanywhere.com · X-API-KEY: ironmind_secret_2026

19

Endpoints

40

Tests Passing

6

Citations

1K

Records

4

DB Tables

19

Endpoints

Across 8 resource modules

40

Tests Passing

Integration test suite

6

Citations

Peer-reviewed sources

1,000

Dataset Records

Kaggle athlete data

4

DB Tables

Athletes / Sessions / Sleep / Check-ins

A

Problem Statement & Purpose

Modern endurance athletes generate vast data across platforms, yet training readiness — defined as the balance between accumulated training stress and recovery — is rarely quantified automatically. Research confirms that sleep quality and wellness signals are directly linked to endurance performance (Roberts et al., 2019; Nuuttila et al., 2025; Matos et al., 2025). The IronMind Coach API addresses this gap by providing a centralised, evidence-based REST service that transforms raw training and wellness logs into actionable decision-support metrics for coaches and self-coached triathletes.

B

Academic Foundation & Scientific Accuracy

Every algorithmic decision maps directly to a peer-reviewed source. The API does not use arbitrary thresholds — each modifier is grounded in sports science literature:

Algorithmic Decision	Scientific Basis	Citation(s)
RPE × duration load proxy	Session-RPE is a validated, practical endurance load metric	Braun-Trocchio et al. (2022); Nuuttila et al. (2025)
ACWR 0.8–1.3 = optimal zone	7-day acute vs. 28-day chronic ratio minimises injury risk	Nuuttila et al. (2025); Matos et al. (2025)
Sleep ≥ 8 h = +10 readiness pts	Sleep duration is a primary determinant of recovery	Roberts et al. (2019); Matos et al. (2025)
Temp > 30 °C = −20 readiness pts	Core temperature rise impairs endurance & raises injury risk	Racinais et al. (2015)
Three-tier stateless REST design	Statelessness supports scalability; layered separation aids maintainability	Li & Chou (2011); Bernstein (1996)
Relational schema + FK constraints	Referential integrity is fundamental to analytics reliability	Ge (2021); Xing (2023)

C

Data Accuracy & Validation Rigour

Dataset Validation

1,000 real athlete records (Rajak, 2024) validated ACWR under realistic load distributions. ETL normalised training hours to minutes and mapped recovery indices to the wellness schema.

Pydantic v2 Boundary Guards

RPE bounded 1–10, sleep hours 0–24, age ≥ 10. Corrupt data cannot reach rolling-average calculations or bias readiness scores.

Daily Aggregation Guard

Service layer sums multiple sessions per day before computing the 7-day ACWR window, preventing score inflation for twice-daily training. Verified by test T-09.

Duplicate Log Prevention

Composite unique indexes on (athlete_id, date) in sleep and check-in tables prevent duplicate entries that would skew rolling averages. Enforced at DB and API layer (409).

Critical Fatigue Floor

ACWR > 2.0 triggers a hardened score floor, preventing misleadingly high readiness outputs under extreme overload — identified via structured GenAI audit.

40 Real-Database Tests

Each test spins up an isolated transient SQLite database and fires live HTTP requests. No ORM mocking — catches constraint violations, cascade deletes, and ACWR edge cases.

D

Technology Stack & Architecture Justification

FastAPI 0.128

Async + auto OpenAPI generation (Li & Chou, 2011)

SQLAlchemy 2.0

DB-agnostic ORM; migrate via one .env change

Pydantic v2

Annotated constraints; invalid data rejected at gateway

JWT + RBAC

python-jose HS256; Passlib bcrypt; role-based access

slowapi

120 req/min global; 30 req/min on /health

OpenWeatherMap

Live heat/humidity fetch; mock fallback if no key

pytest + httpx

Isolated transient DB per session; real HTTP calls

PythonAnywhere

Live WSGI deployment; .env secrets gitignored

E

Academic References

Bernstein, B.E. (1996)

'Three-Tier Client/Server Computing', Proc. FAIM 1996, pp. 1–8.

Ge, M. (2021)

'Research on Relational Databases and SQL', Journal of Physics: Conf. Series, 1848(1), 012095.

Matos et al. (2025)

'Influence of Sleep Quality on Recovery and Performance in Endurance Runners', Healthcare, 13(4), 456.

OpenWeather (2026)

Current Weather Data API. Available at: openweathermap.org/api [Accessed: 11 March 2026].

Rajak, P. (2024)

Athlete Training and Recovery Tracker Dataset. Kaggle. [Accessed: 4 March 2026].

Xing, B. (2023)

'Research on Web Application Performance Optimisation', Highlights in Science, 39, pp. 360–366.

Braun-Trocchio et al. (2022)

'Recovery Strategies in Endurance Athletes', Journal of Functional Morphology and Kinesiology, 7(1), 16.

Li & Chou (2011)

'Design of REST API without Violating REST', Proc. IEEE ICWS 2011, pp. 508–515.

Nuuttila et al. (2025)

'Monitoring Sleep and Nightly Recovery with Wrist-Worn Wearables', Sensors, 25(1), 123.

Racinais et al. (2015)

'Consensus Recommendations on Training and Competing in the Heat', BJSM, 49(18), pp. 1164–1173.

Roberts et al. (2019)

'Assessing Sleep in Athletes: A Systematic Review', Sleep Medicine Reviews, 45, pp. 1–16.

1. Authentication

IronMind uses a hybrid security model. Obtain a JWT Bearer token for protected endpoints, or use the master API key for script/dashboard access.

1.1 Obtain a JWT Token

POST /api/v1/auth/login — Public endpoint, no auth required.

Request body (OAuth2 form-data):

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded

username=athlete@example.com&password;=yourpassword
```

Success Response **200 OK**:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

1.2 Using Auth in Requests

Method	Header / Param	Value	Use Case
JWT Bearer	Authorization	Bearer	Standard protected endpoints
Master Key (header)	X-API-KEY	ironmind_secret_2026	Dashboard / scripts
Master Key (query)	?api_key=...	ironmind_secret_2026	Legacy compat.

1.3 Permission Levels

Access Level	Endpoints
Public	POST /athletes/ (registration), POST /auth/login
JWT / Key	All analytics, CRUD, export, MCP, prescription, coach roster
Coach role only	GET /api/v1/athletes/roster (requires role=coach in JWT)

2. Endpoint Reference Overview

Method	Path	Auth	Description
POST	/api/v1/auth/login	Public	Obtain JWT access token
POST	/api/v1/athletes/	Public	Register new athlete
GET	/api/v1/athletes/	JWT/Key	List all athletes (paginated)

Method	Path	Auth	Description
GET	/api/v1/athletes/{id}	JWT/Key	Get athlete profile
PUT	/api/v1/athletes/{id}	JWT/Key	Update athlete profile
DELETE	/api/v1/athletes/{id}	JWT/Key	Delete athlete + cascade
POST	/api/v1/sessions/	JWT/Key	Log a training session
GET	/api/v1/sessions/	JWT/Key	List sessions (filter by athlete)
POST	/api/v1/sleep-logs/	JWT/Key	Record nightly recovery
POST	/api/v1/checkins/	JWT/Key	Submit wellness check-in
GET	/api/v1/athletes/{id}/insights/readiness	JWT/Key	Real-time readiness score (ACWR)
GET	/api/v1/athletes/{id}/analytics/trends	JWT/Key	14-day load trend array
GET	/api/v1/athletes/{id}/training-prescription	JWT/Key	Training tier recommendation
GET	/api/v1/athletes/{id}/injury-risk	JWT/Key	ACWR-based injury risk flag
POST	/api/v1/athletes/{id}/whatif/readiness	JWT/Key	What-If score simulator
GET	/api/v1/athletes/{id}/export	JWT/Key	Download training history CSV
GET	/api/v1/athletes/roster	JWT/Key	Coach roster (coach role req.)
GET	/api/v1/mcp/tools	JWT/Key	MCP tool definitions for LLM agents
GET	/api/v1/health	Public	Service health check

3. Athlete Management

POST /api/v1/athletes/ — Register Athlete

Creates a new athlete account. The email must be unique across the system.

Request Body:

```
{
  "name": "Mark Allen", // string, required, non-empty
  "email": "mark@example.com", // valid email, unique
  "age": 35, // integer, min: 10
  "city": "Leeds", // string, used for weather API
  "password": "securepassword123" // string, min 8 chars
}
```

Success Response **201 Created**:

```
{
  "id": 1001,
  "name": "Mark Allen",
  "email": "mark@example.com",
  "age": 35,
  "city": "Leeds",
  "role": "athlete",
  "created_at": "2026-03-04T23:54:00Z"
}
```

Error codes: **400** email already taken | **422** invalid age or email format

GET /api/v1/athletes/ — List Athletes

Returns a paginated list of all athletes. Query params: `?skip=0` (default 0) and `?limit=100` (default 100, max 100).

GET /api/v1/athletes/{id} — Get Athlete

Returns full profile for one athlete. Path param: `id` (integer).

Error codes: **404** athlete not found

PUT /api/v1/athletes/{id} — Update Athlete

Partial update — supply only the fields you want to change. Supports: name, email, age, city.

```
{ "city": "Manchester", "age": 36 } // Only changed fields required
```

DELETE /api/v1/athletes/{id} — Delete Athlete

Permanently removes the athlete and **cascades** — all associated sessions, sleep logs, and check-ins are also deleted. Returns **204 No Content** on success.

Error codes: **404** athlete not found

4. Training & Wellness Logs

POST /api/v1/sessions/ — Log Training Session

Records one workout. The load proxy used in $ACWR = RPE \times \text{duration}$.

```
{
  "athlete_id": 1001,
  "date": "2026-03-10", // ISO 8601 date string
  "sport": "Run", // "Run" | "Bike" | "Swim"
  "duration": 75.0, // float, min: 1.0 (minutes)
  "intensity": 7, // int, 1-10 (RPE scale)
  "distance": 12.5 // float, optional (km)
}
```

Success **201 Created**. Error codes: **404** athlete not found | **422** validation failure

POST /api/v1/sleep-logs/ — Record Sleep

Logs one night of recovery. Constraint: one entry per athlete per date.

```
{
  "athlete_id": 1001,
  "date": "2026-03-10",
  "sleep_hours": 7.5, // float, min: 0.0, max: 24.0
  "sleep_quality": 4 // int, 1 (poor) - 5 (excellent)
}
```

Error codes: **409 Conflict** duplicate log for same date | **422** out-of-range values

POST /api/v1/checkins/ — Wellness Check-In

Captures daily subjective wellness. One entry per athlete per date.

```
{
  "athlete_id": 1001,
  "date": "2026-03-10",
  "fatigue": 6, // int, 1-10 (1=fresh, 10=exhausted)
  "stress": 4, // int, 1-10
  "mood": 7, // int, 1-10
  "soreness": 5 // int, 1-10
}
```

Error codes: **409 Conflict** duplicate check-in | **422** out-of-range values

5. Performance Analytics

GET /api/v1/athletes/{id}/insights/readiness — Readiness Score

Computes a real-time 0–100 readiness score by combining ACWR training load, sleep quality, wellness check-ins, and live weather data from OpenWeatherMap.

```
// Success Response 200 OK
{
  "athlete_id": 1001,
  "date": "2026-03-10",
  "readiness_score": 82,
  "readiness_band": "High",
  "signals": {
    "acwr": 1.1, // 7-day / 28-day load ratio
    "sleep_hours": 8.5,
    "sleep_quality": 4,
    "temperature_c": 18.0,
    "humidity_pct": 55
  },
  "top_reasons": [
    { "reason": "Optimal training load balance", "impact": +15 },
    { "reason": "Good sleep duration", "impact": +10 }
  ]
}
```

Bands: **High** (70–100) | **Medium** (40–69) | **Low** (0–39)

GET /api/v1/athletes/{id}/analytics/trends — 14-Day Load Trends

Returns a chronological 14-day array of daily training load values, used by the SPA dashboard Chart.js visualisation.

```
// Success Response 200 OK
{
  "athlete_id": 1001,
  "period_days": 14,
  "trend": [
    { "date": "2026-02-25", "load": 420, "sport": "Run" },
    { "date": "2026-02-26", "load": 0, "sport": null },
    { "date": "2026-02-27", "load": 560, "sport": "Bike" },
    ...
  ]
}
```

GET /api/v1/athletes/{id}/training-prescription — Training Tier

Maps current ACWR to one of four evidence-based training tiers with specific guidance.

ACWR Zone	Tier	Guidance
< 0.5	Rest	Severely deconditioned. Full rest day recommended.
0.5 – 0.79	Recover	Below optimal. Light active recovery only.
0.8 – 1.3	Maintain	Sweet spot. Maintain current training load.
> 1.3	Build	Overreaching risk. Reduce volume immediately.

```
// Example Response 200 OK
{
  "athlete_id": 1001,
  "acwr": 1.1,
  "tier": "Maintain",
  "guidance": "ACWR is in the optimal range. Maintain current training load.",
  "acwr_zone": "sweet_spot"
}
```

GET /api/v1/athletes/{id}/injury-risk — Injury Risk Flag

Returns a binary risk flag and contributing factors based on ACWR threshold analysis.

```
// Example Response 200 OK
{
  "athlete_id": 1001,
  "risk_flag": true,
  "acwr": 1.62,
  "risk_level": "High",
  "factors": ["ACWR exceeds 1.5 danger threshold"]
}
```

6. Advanced Features

POST /api/v1/athletes/{id}/whatif/readiness — What-If Simulator

Projects a future readiness score by recalculating ACWR as if a planned session and sleep had already occurred. Useful for session planning without committing data.

Request Body:

```
{
  "planned_session_duration": 90, // minutes
  "planned_session_intensity": 8, // RPE 1-10
  "expected_sleep_hours": 8.5,
  "expected_sleep_quality": 4 // 1-5
}
```

Response **200 OK**:

```
{
  "athlete_id": 1001,
  "original_score": 68,
  "projected_score": 79,
  "score_change": +11,
  "change_description": "Projected readiness improves with adequate sleep.",
  "projected_acwr": 1.08
}
```

GET /api/v1/athletes/{id}/export — Training History CSV

Streams complete training history as a downloadable CSV file. No request body required.

Response headers:

```
Content-Type: text/csv; charset=utf-8
Content-Disposition: attachment; filename="athlete_1001_training.csv"
```

CSV Schema:

Column	Type	Description
Date	date	Session date (YYYY-MM-DD)
Sport	string	Run Bike Swim
Duration_Min	float	Session duration in minutes
Intensity_RPE	int	Rate of Perceived Exertion (1–10)
Distance_KM	float	Distance covered (km, may be empty)

GET /api/v1/athletes/roster — Coach Roster

Returns all athletes sorted by current readiness score descending. Requires **role=coach** in the JWT payload — returns **403 Forbidden** for athletes.

```
// Example Response 200 OK
{
```

```
"roster": [  
  { "athlete_id": 1001, "name": "Mark Allen", "readiness_score": 82, "band": "High" },  
  { "athlete_id": 1003, "name": "Jane Doe", "readiness_score": 61, "band": "Medium" }  
,  
  "band_summary": { "High": 1, "Medium": 1, "Low": 0 }  
,  
  }
```

GET /api/v1/mcp/tools — MCP Tool Definitions

Returns standardised JSON function definitions conforming to the Model Context Protocol (MCP). Allows LLM-based AI coaches to autonomously call IronMind endpoints without manual integration.

```
// Abbreviated example – three of the available tool definitions  
{  
  "tools": [  
    {  
      "name": "get_readiness_score",  
      "description": "Returns real-time readiness for an athlete",  
      "parameters": { "athlete_id": { "type": "integer", "required": true } }  
    },  
    {  
      "name": "simulate_whatif",  
      "description": "Projects future readiness given planned session + sleep",  
      "parameters": { "athlete_id": "integer", "planned_duration": "float", ... }  
    },  
    { "name": "log_session", ... }  
  ]  
}
```

7. The Readiness Algorithm

The readiness score is a 0–100 integer computed by a weighted modifier system anchored to sports-science evidence. The primary driver is the Acute:Chronic Workload Ratio (ACWR).

7.1 ACWR Formula

Load = RPE (1–10) multiplied by session duration (minutes). ACWR is the ratio of acute (7-day sum) to chronic (28-day average daily) load:

```
ACWR = Sum_of_Load(last 7 days) / Avg_Daily_Load(last 28 days)
```

```
Optimal range: 0.8 – 1.3 ("sweet spot")
```

```
Danger zone: > 1.5 (injury risk)
```

7.2 Score Modifiers

Signal	Condition	Modifier	Rationale
Baseline	Always	+50 pts	Starting point
ACWR (sweet spot)	0.8 – 1.3	+15 pts	Optimal training stimulus
ACWR (danger)	> 1.5	–20 pts	Injury risk zone
ACWR (deconditioning)	< 0.5	–10 pts	Under-trained
Sleep duration	>= 8 hours	+10 pts	Full recovery achieved
Sleep quality	Quality >= 4/5	+5 pts	High quality sleep
Sleep deficit	< 6.5 hours	–15 pts	Chronic deprivation
Heat stress	Temp > 30°C	–20 pts	Racinais et al. (2015)
Humidity stress	Humidity > 80%	–10 pts	Compound heat burden

Final score is clamped to 0–100. Environmental data is fetched live from OpenWeatherMap using the athlete's city field. A mock fallback is used when no API key is configured.

8. Pydantic Validation Rules

All input is validated by Pydantic v2 before reaching the database. Violations return a **422 Unprocessable Entity** with field-level error details.

Field	Type	Constraint	Error on Violation
name	str	Non-empty string	422 — blank name rejected
email	str	Valid RFC 5322 email	422 — invalid format
age	int	min: 10	422 — too young
password	str	min length: 8	422 — too short
sport	str	Run Bike Swim	422 — unknown sport
duration	float	min: 1.0 (minutes)	422 — non-positive duration
intensity (RPE)	int	1 – 10	422 — out of RPE scale

Field	Type	Constraint	Error on Violation
sleep_hours	float	0.0 – 24.0	422 — impossible value
sleep_quality	int	1 – 5	422 — out of range
fatigue	int	1 – 10	422 — out of range
stress	int	1 – 10	422 — out of range
mood	int	1 – 10	422 — out of range
soreness	int	1 – 10	422 — out of range

9. Error Reference

9.1 Standard Error Envelope

All error responses use the same JSON structure to ensure consistent client handling.

```
{
  "error": {
    "status_code": 422,
    "type": "validation_error",
    "message": "Input validation failed",
    "details": [
      { "field": "age", "msg": "must be >= 10" }
    ]
  }
}
```

9.2 HTTP Status Code Reference

Code	Type	Triggered By
200	OK	Successful GET / analytics request
201	Created	Successful POST (athlete, session, sleep, check-in)
204	No Content	Successful DELETE
400	bad_request	Business rule violation (e.g. email already registered)
401	unauthorized	Missing or expired JWT Bearer token
403	forbidden	Valid token but insufficient role (e.g. non-coach accessing roster) or invalid X-API-KEY
404	not_found	Athlete or resource ID does not exist
409	conflict	Unique constraint violation (duplicate sleep log or check-in for same date)
422	validation_error	Pydantic field constraint failed (type, range, format)
429	rate_limited	Exceeded 120 req/min (slowapi)
500	internal_error	Unhandled server exception

9.3 Example Error Responses

404 Not Found

```
{ "error": { "status_code": 404, "type": "not_found", "message": "Athlete not found" } }
```

409 Conflict (duplicate sleep log)

```
{ "error": { "status_code": 409, "type": "conflict",
  "message": "Sleep log already exists for this athlete on 2026-03-10" } }
```

403 Forbidden (missing API key)

```
{ "error": { "status_code": 403, "type": "unauthorized",
  "message": "Missing or invalid authentication credentials" } }
```

For interactive testing visit the Swagger console at <https://osamakadhem.pythonanywhere.com/docs> or the ReDoc reference at [/redoc](#). All endpoints can be tested with the master key: X-API-KEY: ironmind_secret_2026